The topology of risk in scientific software

5. Expert questionnaire validation

Arnald Puy

Contents

1 Preliminary

```r
# PRELIMINARY FUNCTIONS ########################################################
################################################################################

sensobol::load_packages(c("data.table", "readxl", "ggplot2", "cowplot",
                          "softwareRisk", "parallel", "benchmarkme"))

theme_AP <- softwareRisk::theme_AP

# Load data --------------------------------------------------------------------

tidy <- as.data.table(read_excel("./questionnaire_data/questionnaire_analysis.xlsx",
                                 sheet = "1_TidyData"))

setnames(tidy,
  old = seq_len(14),
  new = c("model", "respondent_id", "path_num", "path_name",
          "path_type", "familiarity_text", "familiarity_num",
          "q1_real_execution", "q2_central_workflow",
          "q3_maintenance", "q4_system_consequences",
          "mean_likert", "excluded_unfamiliar", "comment"))

usable <- tidy[excluded_unfamiliar == FALSE]

usable[, path_type := factor(path_type, levels = c("high_risk", "low_risk"),
                             labels = c("High-risk", "Low-risk"))]
usable[, model := factor(model, levels = c("CTSM","H08","HyrdroPy","HYPE","PCRGLOBWB"),
                             labels = c("CTSM","H08","HydroPy","HYPE","PCR-GLOBWB"))]

q_cols  <- c("q1_real_execution","q2_central_workflow",
             "q3_maintenance","q4_system_consequences")
q_names <- c("Q1: Real execution chain",
             "Q2: Central to workflow",
             "Q3: Maintenance relevance",
             "Q4: System-wide consequences")
```

2 Sample summary

```r
# SAMPLE #######################################################################

cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```r
cat("SAMPLE\n")
```

```
## SAMPLE
```

```r
cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```r
cat("Total records (all)  :", nrow(tidy), "\n")
```

```
## Total records (all)  : 107
```

```r
cat("Excluded (unfamiliar):", nrow(tidy[excluded_unfamiliar == TRUE]), "\n")
```

```
## Excluded (unfamiliar): 18
```

```r
cat("Usable records       :", nrow(usable), "\n\n")
```

```
## Usable records       : 87
```

```r
print(usable[, .N, by = .(model, path_type)])
```

```
##          model path_type     N
##         <fctr>    <fctr> <int>
##   1:      CTSM High-risk    23
##   2:      CTSM  Low-risk     4
##   3:       H08 High-risk    10
##   4:       H08  Low-risk     5
##   5:  HydroPy High-risk    10
##   6:  HydroPy  Low-risk     5
##   7:      HYPE High-risk    10
##   8:      HYPE  Low-risk     5
##   9: PCR-GLOBWB High-risk    10
##  10: PCR-GLOBWB  Low-risk     5
```

3 Descriptive statistics

```r
# DESCRIPTIVE STATISTICS ########################################################

cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```r
cat("DESCRIPTIVES: mean_likert (all models pooled)\n")
```

```
## DESCRIPTIVES: mean_likert (all models pooled)
```

```r
cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```r
desc <- usable[, .(
  N      = sum(!is.na(mean_likert)),
  Mean   = round(mean(mean_likert, na.rm = TRUE), 3),
  Median = round(median(mean_likert, na.rm = TRUE), 3),
  SD     = round(sd(mean_likert, na.rm = TRUE), 3),
```

```r
  Q25    = round(quantile(mean_likert, .25, na.rm = TRUE), 3),
  Q75    = round(quantile(mean_likert, .75, na.rm = TRUE), 3)
), by = path_type]
print(desc)
```

```
##    path_type     N  Mean Median    SD   Q25   Q75
##       <fctr> <int> <num>  <num> <num> <num> <num>
## 1: High-risk    63 3.187    3.5 1.063   2.5 4.000
## 2:  Low-risk    23 3.304    3.5 0.482   3.0 3.625
```

```r
cat("\n")
```

```r
cat("Per-question means by path type:\n")
```

```
## Per-question means by path type:
```

```r
for (i in seq_along(q_cols)) {
  hr <- usable[path_type == "High-risk",  get(q_cols[i])]
  lr <- usable[path_type == "Low-risk",   get(q_cols[i])]
  cat(sprintf("  %-35s  High = %.3f  Low = %.3f  Diff = %+.3f\n",
              q_names[i], mean(hr, na.rm=T), mean(lr, na.rm=T),
              mean(hr, na.rm=T) - mean(lr, na.rm=T)))
}
```

```
##   Q1: Real execution chain             High = 3.460  Low = 3.783  Diff = -0.322
##   Q2: Central to workflow              High = 3.556  Low = 3.391  Diff = +0.164
##   Q3: Maintenance relevance            High = 2.413  Low = 2.522  Diff = -0.109
##   Q4: System-wide consequences         High = 3.317  Low = 3.522  Diff = -0.204
```

4 Primary test

```r
# PRIMARY TEST #################################################################

cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```r
cat("PRIMARY: Wilcoxon rank-sum, mean_likert, all models pooled\n")
```

```
## PRIMARY: Wilcoxon rank-sum, mean_likert, all models pooled
```

```r
cat("H1: high-risk > low-risk (one-sided)\n")
```

```
## H1: high-risk > low-risk (one-sided)
```

```r
cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```r
hr_all <- usable[path_type == "High-risk", mean_likert]
lr_all <- usable[path_type == "Low-risk",  mean_likert]
```

```
wt_all <- wilcox.test(hr_all, lr_all, alternative = "greater", exact = FALSE)
A_all  <- wt_all$statistic / (length(hr_all) * length(lr_all))

cat(sprintf("  W = %.0f\n  p = %.4f (one-sided, high > low)\n  A = %.3f\n",
            wt_all$statistic, wt_all$p.value, A_all))
```

```
##   W = 736
##   p = 0.4589 (one-sided, high > low)
##   A = 0.486
```

```
cat(sprintf("  n_high = %d, n_low = %d\n", length(hr_all), length(lr_all)))
```

```
##   n_high = 63, n_low = 24
```

5 Per-question tests

```
# PER-QUESTION TESTS ############################################################

cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```
cat("PER-QUESTION Wilcoxon rank-sum (one-sided), all models\n")
```

```
## PER-QUESTION Wilcoxon rank-sum (one-sided), all models
```

```
cat("-----------------------------------------------------------------\n")
```

```
## -----------------------------------------------------------------
```

```
pq_results <- lapply(seq_along(q_cols), function(i) {
  hr <- usable[path_type == "High-risk", get(q_cols[i])]
  lr <- usable[path_type == "Low-risk",  get(q_cols[i])]
  wt <- wilcox.test(hr, lr, alternative = "greater", exact = FALSE)
  A  <- wt$statistic / (sum(!is.na(hr)) * sum(!is.na(lr)))
  list(q = q_names[i], W = wt$statistic, p = wt$p.value, A = A,
       mean_hr = mean(hr, na.rm=T), mean_lr = mean(lr, na.rm=T))
})

raw_p <- sapply(pq_results, `[[`, "p")
adj_p <- p.adjust(raw_p, method = "BH")

for (i in seq_along(pq_results)) {
  r <- pq_results[[i]]
  cat(sprintf("  %-35s W=%5.0f  p=%.4f  p_BH=%.4f  A=%.3f  Diff=%+.3f\n",
              r$q, r$W, r$p, adj_p[i], r$A, r$mean_hr - r$mean_lr))
}
```

```
##   Q1: Real execution chain            W=  677  p=0.6883  p_BH=0.7351  A=0.467  Diff=-0.322
##   Q2: Central to workflow             W=  811  p=0.1929  p_BH=0.7351  A=0.560  Diff=+0.164
```

```
##   Q3: Maintenance relevance          W=  664  p=0.7351  p_BH=0.7351  A=0.458  Diff=-0.109
##   Q4: System-wide consequences       W=  726  p=0.4980  p_BH=0.7351  A=0.501  Diff=-0.204
```

6 Per-model tests

```
# PER-MODEL TESTS ##############################################################

cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```
cat("PER-MODEL Wilcoxon rank-sum, mean_likert (two-sided)\n")
```

```
## PER-MODEL Wilcoxon rank-sum, mean_likert (two-sided)
```

```
cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```
for (m in levels(usable$model)) {
  sub_m <- usable[model == m]
  hr_m  <- sub_m[path_type == "High-risk", mean_likert]
  lr_m  <- sub_m[path_type == "Low-risk",  mean_likert]
  wt_m  <- wilcox.test(hr_m, lr_m, alternative = "two.sided", exact = FALSE)
  A_m   <- wt_m$statistic / (length(hr_m) * length(lr_m))
  cat(sprintf("  %-12s  W=%5.0f  p=%.4f  A=%.3f  mean_high=%.2f  mean_low=%.2f  n_high=%d  n_lo
              m, wt_m$statistic, wt_m$p.value, A_m,
              mean(hr_m, na.rm=T), mean(lr_m, na.rm=T),
              length(hr_m), length(lr_m)))
}
```

```
##   CTSM          W=   54  p=0.6239  A=0.582  mean_high=3.47  mean_low=3.38  n_high=23  n_low=
##   H08           W=   38  p=0.1204  A=0.760  mean_high=4.00  mean_low=3.55  n_high=10  n_low=
##   HydroPy       W=   28  p=0.7543  A=0.560  mean_high=3.08  mean_low=2.95  n_high=10  n_low=
##   HYPE          W=    0  p=0.0029  A=0.010  mean_high=1.85  mean_low=3.35  n_high=10  n_low=
##   PCR-GLOBWB    W=   24  p=0.6143  A=0.480  mean_high=3.17  mean_low=3.31  n_high=10  n_low=
```

7 Sensitivity: exclude HYPE

```
# SENSITIVITY: EXCLUDE HYPE ####################################################

cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```
cat("SENSITIVITY: Exclude HYPE (deprecated calibration paths)\n")
```

```
## SENSITIVITY: Exclude HYPE (deprecated calibration paths)
```

```
cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```r
no_hype <- usable[model != "HYPE"]
hr_nh   <- no_hype[path_type == "High-risk", mean_likert]
lr_nh   <- no_hype[path_type == "Low-risk",  mean_likert]
wt_nh   <- wilcox.test(hr_nh, lr_nh, alternative = "greater", exact = FALSE)
A_nh    <- wt_nh$statistic / (length(hr_nh) * length(lr_nh))

cat(sprintf("  W = %.0f\n  p = %.4f (one-sided)\n  A = %.3f\n",
            wt_nh$statistic, wt_nh$p.value, A_nh))
```

```
##   W = 588
##   p = 0.0701 (one-sided)
##   A = 0.584
```

```r
cat(sprintf("  n_high = %d, n_low = %d\n", length(hr_nh), length(lr_nh)))
```

```
##   n_high = 53, n_low = 19
```

```r
cat(sprintf("  mean_high = %.3f, mean_low = %.3f\n\n",
            mean(hr_nh, na.rm=T), mean(lr_nh, na.rm=T)))
```

```
##   mean_high = 3.439, mean_low = 3.292
```

```r
cat("Per-question (excluding HYPE):\n")
```

```
## Per-question (excluding HYPE):
```

```r
for (i in seq_along(q_cols)) {
  hr <- no_hype[path_type == "High-risk", get(q_cols[i])]
  lr <- no_hype[path_type == "Low-risk",  get(q_cols[i])]
  wt <- wilcox.test(hr, lr, alternative = "greater", exact = FALSE)
  A  <- wt$statistic / (sum(!is.na(hr)) * sum(!is.na(lr)))
  cat(sprintf("  %-35s p=%.4f  A=%.3f  Diff=%+.3f\n",
              q_names[i], wt$p.value, A, mean(hr,na.rm=T)-mean(lr,na.rm=T)))
}
```

```
##   Q1: Real execution chain            p=0.1669  A=0.572  Diff=+0.089
##   Q2: Central to workflow             p=0.1647  A=0.575  Diff=+0.233
##   Q3: Maintenance relevance           p=0.3615  A=0.527  Diff=+0.086
##   Q4: System-wide consequences        p=0.1193  A=0.589  Diff=+0.180
```

8 Familiarity as moderator

```r
# FAMILIARITY CORRELATION #####################################################

cat("-----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```r
cat("FAMILIARITY CORRELATION with mean_likert\n")
```

```
## FAMILIARITY CORRELATION with mean_likert
```

```r
cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

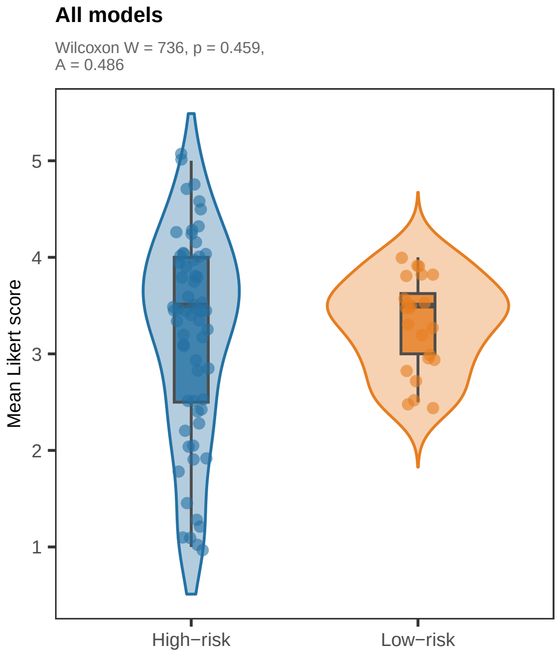
```r
fam_cor <- cor.test(usable$familiarity_num, usable$mean_likert,
                    method = "spearman", exact = FALSE)
cat(sprintf("  Spearman rho = %.3f,  p = %.4f\n",
            fam_cor$estimate, fam_cor$p.value))
```

```
##   Spearman rho = -0.026,  p = 0.8150
```

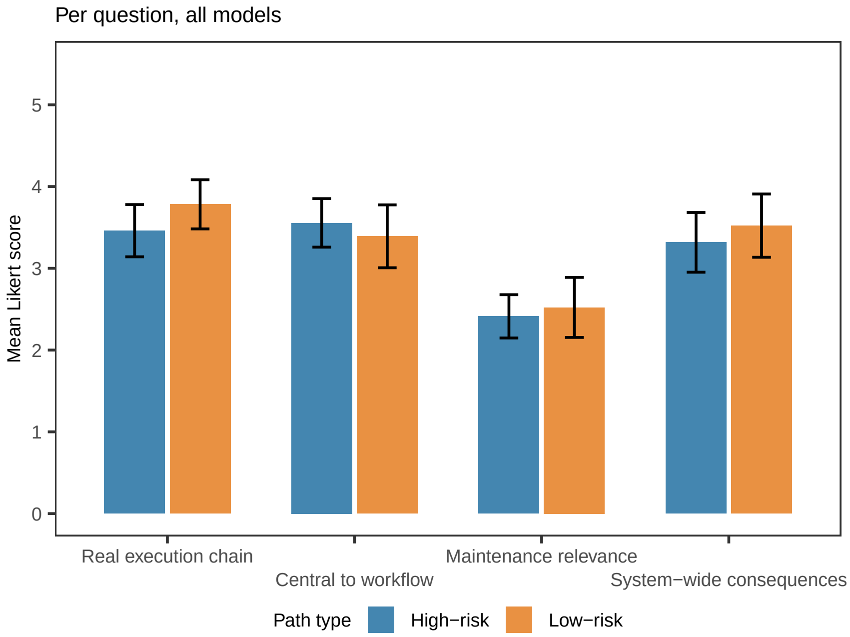
9 Plots

```r
# PLOT DISTRIBUTIONS ############################################################

p1 <- ggplot(usable, aes(x = path_type, y = mean_likert,
                          fill = path_type, colour = path_type)) +
  geom_violin(alpha = 0.35, trim = FALSE, width = 0.8) +
  geom_boxplot(width = 0.15, outlier.shape = NA, alpha = 0.8,
               colour = "grey30") +
  geom_jitter(width = 0.08, size = 1.5, alpha = 0.6) +
  scale_fill_manual(values   = c("High-risk" = "#2471A3",
                                 "Low-risk"  = "#E67E22")) +
  scale_colour_manual(values = c("High-risk" = "#2471A3",
                                 "Low-risk"  = "#E67E22")) +
  scale_y_continuous(limits = c(0.5, 5.5), breaks = 1:5) +
  labs(x = NULL, y = "Mean Likert score",
       title = "All models",
       subtitle = sprintf("Wilcoxon W = %.0f, p = %.3f, \nA = %.3f",
                          wt_all$statistic, wt_all$p.value, A_all)) +
  theme_AP() +
  theme(legend.position = "none",
        plot.title    = element_text(face = "bold"),
        plot.subtitle = element_text(size = 6, colour = "grey40"))

p1
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_ydensity()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).

## Warning: Removed 120 rows containing missing values or values outside the scale range
## (`geom_violin()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```



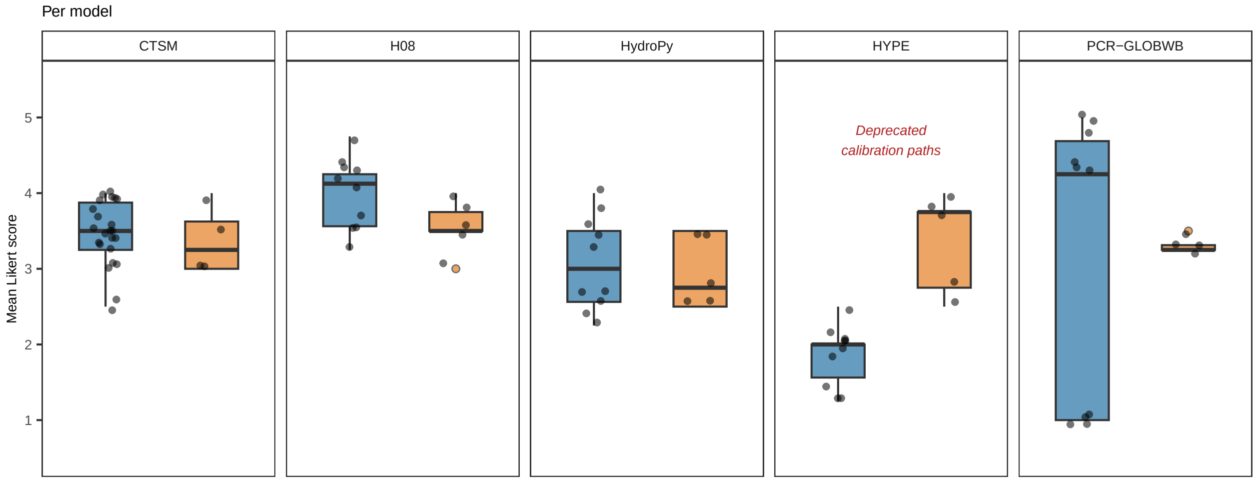

```
# PLOT PER QUESTION ############################################################

q_long <- melt(usable, measure.vars = q_cols,
               variable.name = "question", value.name = "score")
q_long[, question := factor(question, levels = q_cols, labels = q_names)]
q_long <- q_long[!is.na(score)]

q_desc <- q_long[, .(mean  = mean(score),
                      lower = mean(score) - 1.96 * sd(score) / sqrt(.N),
                      upper = mean(score) + 1.96 * sd(score) / sqrt(.N)),
                 by = .(question, path_type)]

p3 <- ggplot(q_desc, aes(x = question, y = mean, fill = path_type,
                          ymin = lower, ymax = upper)) +
  geom_col(position = position_dodge(0.7), width = 0.65, alpha = 0.85) +
  geom_errorbar(position = position_dodge(0.7), width = 0.2,
                linewidth = 0.5) +
  scale_fill_manual(values = c("High-risk" = "#2471A3",
                                "Low-risk"  = "#E67E22"),
                    name = "Path type") +
  scale_y_continuous(limits = c(0, 5.5), breaks = 0:5) +
  scale_x_discrete(labels = function(x) sub("Q\\d+: ", "", x),
                   guide = guide_axis(n.dodge=2)) +
  labs(x = NULL, y = "Mean Likert score",
       title = "Per question, all models") +
  theme_AP() +
  theme(legend.position = "bottom")

p3
```




```
# PLOT PER MODEL ###############################################################

p4 <- ggplot(usable,
             aes(x = path_type, y = mean_likert, fill = path_type)) +
  geom_boxplot(width = 0.5, outlier.shape = 21, outlier.size = 1.5,
               alpha = 0.7) +
  geom_jitter(width = 0.12, size = 1.2, alpha = 0.55) +
  scale_fill_manual(values = c("High-risk" = "#2471A3",
                               "Low-risk"  = "#E67E22"),
                    name = "Path type") +
  scale_y_continuous(limits = c(0.5, 5.5), breaks = 1:5) +
  facet_wrap(~model, nrow = 1) +
  geom_text(data = data.frame(model = factor("HYPE", levels = levels(usable$model)),
                              x = 1.5, y = 4.7),
            aes(x = x, y = y, label = "Deprecated\ncalibration paths"),
            inherit.aes = FALSE,
            size = 2.5, colour = "firebrick", fontface = "italic") +
  labs(x = NULL, y = "Mean Likert score",
       title = "Per model") +
  theme_AP() +
  theme(axis.text.x      = element_blank(),
        axis.ticks.x     = element_blank(),
        legend.position  = "none")

p4
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
```

```
## (`geom_point()`).
```



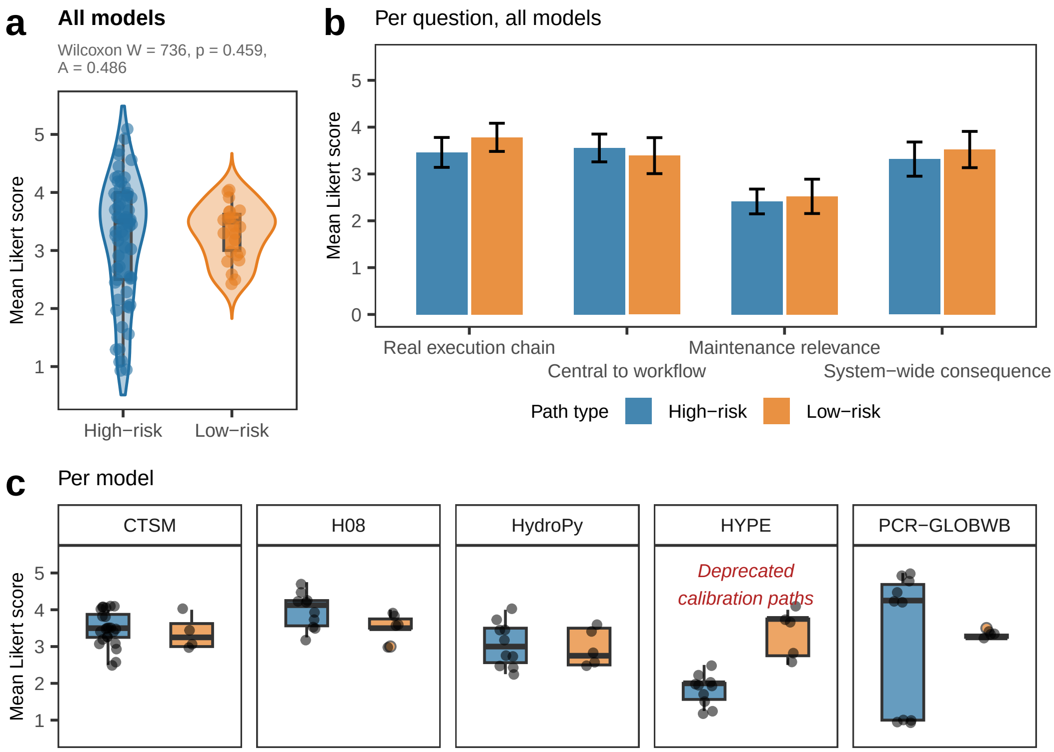

```r
# ASSEMBLE FIGURE ##############################################################

top  <- plot_grid(p1, p3, nrow = 1, rel_widths = c(0.3, 0.7),
                  labels = c("a","b"))
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_ydensity()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).

## Warning: Removed 120 rows containing missing values or values outside the scale range
## (`geom_violin()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```r
full <- plot_grid(top, p4, ncol = 1, rel_heights = c(0.6, 0.4),
                  labels = c("","c"))
```

```
## Warning: Removed 1 row containing non-finite outside the scale range (`stat_boxplot()`).
## Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```r
full
```


a
All models
Wilcoxon W = 736, p = 0.459, A = 0.486
Mean Likert score
High−risk
Low−risk
b
Per question, all models
Mean Likert score
Real execution chain
Central to workflow
Maintenance relevance
System−wide consequence
Path type
High−risk
Low−risk
c
Per model
CTSM
H08
HydroPy
HYPE
PCR−GLOBWB
Deprecated calibration paths
Mean Likert score

10 Session information

```r
# SESSION INFORMATION ##########################################################

sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framew
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] parallel  stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
## [1] benchmarkme_1.0.8   softwareRisk_0.2.0  cowplot_1.2.0.9000
## [4] ggplot2_4.0.1.9000  readxl_1.4.5        data.table_1.18.2.1
##
## loaded via a namespace (and not attached):
##  [1] benchmarkmeData_1.0.4 gtable_0.3.6          xfun_0.55
##  [4] ggrepel_0.9.6         lattice_0.22-7        vctrs_0.7.1
##  [7] tools_4.5.2           Rdpack_2.6.4          generics_0.1.4
## [10] tibble_3.3.1          pkgconfig_2.0.3       Matrix_1.7-4
## [13] RColorBrewer_1.1-3    S7_0.2.1              lifecycle_1.0.5
## [16] ineq_0.2-13           compiler_4.5.2        farver_2.1.2
## [19] tinytex_0.58          ggforce_0.5.0         graphlayouts_1.2.2
## [22] codetools_0.2-20      htmltools_0.5.9       yaml_2.3.12
## [25] pillar_1.11.1         tidyr_1.3.2           MASS_7.3-65
## [28] cachem_1.1.0          iterators_1.0.14      viridis_0.6.5
## [31] foreach_1.5.2         tidyselect_1.2.1      digest_0.6.39
## [34] dplyr_1.1.4           purrr_1.2.1           labeling_0.4.3
## [37] polyclip_1.10-7       sensobol_1.1.6        fastmap_1.2.0
## [40] grid_4.5.2            cli_3.6.5             magrittr_2.0.4
## [43] ggraph_2.2.2          tidygraph_1.3.1       withr_3.0.2
## [46] scales_1.4.0          rmarkdown_2.30        httr_1.4.8
## [49] igraph_2.2.1          otel_0.2.0            gridExtra_2.3
## [52] cellranger_1.1.0      memoise_2.0.1         evaluate_1.0.5
## [55] knitr_1.51            rbibutils_2.4         doParallel_1.0.17
```

```
## [58] viridisLite_0.4.2     rlang_1.1.7          Rcpp_1.1.1
## [61] glue_1.8.0            tweenr_2.0.3         rstudioapi_0.17.1
## [64] R6_2.6.1
```

```r
## Return the machine CPU -------------------------------------------------------

cat("Machine:     "); print(get_cpu()$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```r
## Return number of true cores ------------------------------------------------

cat("Num cores:   "); print(parallel::detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```r
## Return number of threads ---------------------------------------------------

cat("Num threads: "); print(parallel::detectCores(logical = FALSE))
```

```
## Num threads:
```

```
## [1] 10
```